

services\cache\cache_manager.py

```
import threading
import time
from datetime import datetime, timezone, timedelta
from typing import Dict, List, Any, Optional
import json
import config
from database.db_manager import db_manager

class CacheManager:
    """Lightweight caching system with database persistence for reduced memory footprint"""
    def __init__(self):
        # Only cache metadata, not actual CVE data
        self._cache_timestamps = {}
        self._historical_loaded = False
        self._lock = threading.Lock()

    # Cache duration configuration (in seconds)
    self.API_CACHE_DURATION = 24 * 60 * 60 # 24 hours
    self.HISTORICAL_CACHE_DURATION = 7 * 24 * 60 * 60 # 7 days

    # Reference to the database manager
    self.db_manager = db_manager

    # Memory usage monitoring
    self._last_memory_check = time.time()
    self._memory_check_interval = 60 # Check memory usage every minute

    def get_api_data_30_days(self) -> Optional[bool]:
        """Check if API data cache is valid - returns True if DB should be used"""
        cache_key = "api_30_days"

        # Check database cache first
        if self.db_manager.use_database:
            cache_meta = self.db_manager.get_cache_metadata(cache_key)
            if cache_meta:
                last_updated = cache_meta.get('last_updated')
                if last_updated:
                    # Convert to timestamp if datetime
                    if isinstance(last_updated, datetime):
                        timestamp = last_updated.timestamp()
                    else:
                        timestamp = datetime.fromisoformat(str(last_updated).replace('Z', '+00:00')).timestamp()

                    age = datetime.now().timestamp() - timestamp
                    if age < self.API_CACHE_DURATION:
                        print(f"[Cache] Database API cache still valid (age: {int(age/60)} minutes)")
                        return True

        # Fall back to in-memory cache if database not available
        with self._lock:
            if cache_key in self._cache_timestamps:
                timestamp = self._cache_timestamps.get(cache_key, 0)
                age = datetime.now().timestamp() - timestamp
                if age < self.API_CACHE_DURATION:
                    print(f"[Cache] Memory API cache still valid (age: {int(age/60)} minutes)")
                    return True

        return None

    def set_api_data_30_days(self, data: List[Dict[str, Any]]):
        """Set cache timestamp for API data"""
        cache_key = "api_30_days"
        now = datetime.now().timestamp()

        # Update database cache
        if self.db_manager.use_database:
            self.db_manager.update_cache_metadata(
                cache_key,
                len(data),
                {"last_check": now, "type": "api_30_days"}
            )
```

```

# Update memory cache
with self._lock:
self._cache_timestamps[cache_key] = now

print(f"[Cache] Set timestamp for API data ({len(data)} CVEs)")

def get_historical_data(self) -> Optional[bool]:
    """Check if historical data is loaded in the database"""
    # Check database first
    if self.db_manager.use_database:
        cache_meta = self.db_manager.get_cache_metadata("historical_loaded")
        if cache_meta:
            print("[Cache] Historical data marked as loaded in database")
            return True

    # Fall back to in-memory flag
    if self._historical_loaded:
        print("[Cache] Historical data marked as loaded in memory")
        return True

    return None

def set_historical_data(self, data: Dict[str, List[Dict]]):
    """Mark historical data as loaded"""
    total_cves = sum(len(year_data) for year_data in data.values())

    # Update database cache
    if self.db_manager.use_database:
        self.db_manager.update_cache_metadata(
            "historical_loaded",
            total_cves,
            {"years": list(data.keys())}
        )

    # Update memory flag
    with self._lock:
        self._historical_loaded = True

    print(f"[Cache] Marked historical data as loaded ({total_cves} CVEs)")

def get_summary_stats(self, key: str) -> Optional[Dict[str, Any]]:
    """Get cached summary statistics"""
    if self.db_manager.use_database:
        return self.db_manager.get_summary_stats(key)
    return None

def set_summary_stats(self, key: str, count: int, data: Dict[str, Any]):
    """Set cached summary statistics"""
    if self.db_manager.use_database:
        self.db_manager.save_summary_stats(key, count, data)
    print(f"[Cache] Saved summary stats for {key} ({count} records)")

def get_timeline_data(self, years=1) -> Optional[Dict[str, Any]]:
    """Get cached timeline data"""
    if self.db_manager.use_database:
        return self.db_manager.get_timeline_data(years)
    return None

def set_timeline_data(self, data: Dict[str, Any]):
    """Save timeline data"""
    if self.db_manager.use_database and 'raw_data' in data:
        year_month_counts = {}

        for date_str, count in data['raw_data'].items():
            parts = date_str.split('-')
            if len(parts) >= 2:
                year = int(parts[0])
                month = int(parts[1])
                year_month_counts[(year, month)] = count

        self.db_manager.save_timeline_data(year_month_counts)
        print(f"[Cache] Saved timeline data ({len(year_month_counts)} records)")

def get_cache_stats(self) -> Dict[str, Any]:
    """Get cache statistics"""
    with self._lock:
        stats = {

```

```

'cached_timestamps': len(self._cache_timestamps),
'historical_loaded': self._historical_loaded,
'cache_ages': {},
'database_enabled': self.db_manager.use_database
}

current_time = datetime.now().timestamp()
for key, timestamp in self._cache_timestamps.items():
    age_minutes = int((current_time - timestamp) / 60)
    stats['cache_ages'][key] = f"{age_minutes} minutes"

# Add database stats if available
if self.db_manager.use_database:
    db_stats = self.db_manager.get_stats()
    stats.update(db_stats)

return stats

def clear_cache(self):
    """Clear all cached data"""
    # Clear in-memory cache
    with self._lock:
        self._cache_timestamps.clear()
        self._historical_loaded = False

    # Clear database cache if available
    if self.db_manager.use_database:
        self.db_manager.clear_all_cves()

print("[Cache] All caches cleared")

def check_memory_usage(self):
    """Check memory usage and clear cache if needed - NEW"""
    # Only check periodically to reduce overhead
    current_time = time.time()
    if current_time - self._last_memory_check < self._memory_check_interval:
        return

    self._last_memory_check = current_time

    try:
        import psutil
        import os

        process = psutil.Process(os.getpid())
        memory_info = process.memory_info()
        memory_mb = memory_info.rss / 1024 / 1024

        print(f"[Cache] Current memory usage: {memory_mb:.2f} MB")

        # If memory usage is getting too high, clear in-memory caches
        if memory_mb > config.MAX_MEMORY_USAGE_MB:
            print(f"[Cache] WARNING: Memory usage ({memory_mb:.2f} MB) exceeds limit ({config.MAX_MEMORY_USAGE_MB} MB)")
            print("[Cache] Clearing in-memory caches to reduce memory usage")

            with self._lock:
                # Only clear memory caches, keep database intact
                self._cache_timestamps.clear()
                self._historical_loaded = False

            # Force garbage collection
            import gc
            gc.collect()

            # Check memory again
            memory_info = process.memory_info()
            memory_mb_after = memory_info.rss / 1024 / 1024
            print(f"[Cache] Memory usage after cleanup: {memory_mb_after:.2f} MB (freed {memory_mb - memory_mb_after:.2f} MB)")

        except ImportError:
            print("[Cache] psutil not available, skipping memory check")
        except Exception as e:
            print(f"[Cache] Error checking memory: {e}")

    def warm_up_cache(self):

```

```
"""Warm up cache - load basic data into database"""
if self.db_manager.use_database:
    print("[Cache] Performing cache warm-up with database")

    # Nothing to do here - we'll load data on demand
    # This is just a placeholder for future optimization

    return True
else:
    print("[Cache] Cache warm-up skipped (using memory)")
    return False

# Global cache instance
cache_manager = CacheManager()
```